

Rapport de Projet de Fin d'Études Titre :DWWM

**Conception et développement d'une application web de
gestion des services de Maison Yara**

**Projet réalisé dans le cadre de la présentation au Titre Professionnel Développeur Web et Web
Mobile - DWWM**

Présenté par : Khouloud Rais

Encadrant : Sambeau Prak

Promotion CDPI - 2025-2026

Organisme de formation Ecole LAPLATEFORME Cannes

Chapitre 1 : Introduction générale
Chapitre 2 : Présentation de l'entreprise
Chapitre 3 : Étude de l'existant
Chapitre 4 : Analyse des besoins
Chapitre 5 : Cahier des charges fonctionnel et technique
Chapitre 6 : Conception générale
Chapitre 7 : Conception détaillée (MERISE et UML)
Chapitre 8 : Réalisation de l'application
Chapitre 9 : Tests, validation et déploiement Conclusion
générale et perspectives

CHAPITRE 1 : INTRODUCTION GÉNÉRALE

1.1. La transformation numérique a rendu la présence en ligne indispensable pour toute entreprise : visibilité, accessibilité des services, automatisation, meilleure expérience client.

Application au secteur événementiel

Ce secteur illustre bien cette évolution : les clients veulent voir les réalisations passées, des prestations personnalisées, et des démarches simplifiées (réservation, demande d'infos). Une plateforme web devient donc un **outil stratégique** pour renforcer l'image professionnelle d'un prestataire événementiel.

Positionnement du projet

Ce constat justifie le projet réalisé pour **Maison Yara by Berriri**, entreprise d'organisation d'événements privés et professionnels sur la Côte d'Azur — le projet s'inscrit comme réponse concrète à ce besoin de digitalisation du secteur.

1.2. Contexte général du projet

Maison Yara (mariages, anniversaires, baby showers, événements pro, décoration sur mesure, location de matériel) gère aujourd'hui tout manuellement — présentation des services, demandes clients, suivi du matériel — ce qui fait perdre du temps et complique l'organisation interne.

Problématique

Comment concevoir une application web permettant à Maison Yara d'améliorer sa visibilité numérique, de faciliter la relation client et d'optimiser la gestion de ses activités ?

Les freins actuels identifiés :

- visibilité numérique faible
- pas de vitrine pour les réalisations
- pas de gestion structurée des devis
- pas de suivi des réservations
- pas d'organisation du matériel disponible à la location

Objectifs

- *Général* : plateforme web pro pour présenter les services, améliorer la communication client, faciliter devis/réservations, mieux gérer les contenus en interne.
- *Spécifiques* : site vitrine, galerie de réalisations, catalogue de location, formulaire de devis en ligne, espace admin sécurisé, gestion des contenus/images/demandes, sécurité des données, architecture évolutive.

Solution proposée

Application full-stack :

- **Front** : React.js

- **Back** : Node.js + Express.js (API REST)
- **BDD** : MySQL
- Modules : interface publique, services, galerie, formulaire de devis « intelligent », catalogue de location, espace admin sécurisé, gestion dynamique des contenus.

Méthodologie

4 étapes : étude préalable (besoins/existant) → conception (architecture, BDD, UML) → implémentation (front/back/BDD) → tests et validation.

CHAPITRE 2 : PRÉSENTATION DE L'ENTREPRISE

Qui est l'entreprise

- Basée à **Antibes**, intervient sur **Antibes, Cannes, Nice, Monaco**.
- Spécialisée dans la décoration et l'organisation d'événements privés et professionnels, avec une approche créativité / écoute / personnalisation.
- Clientèle mixte : particuliers et professionnels.

Domaines d'activité (3 catégories)

1. **Mariages** — thème décoratif, ambiance, mise en place, coordination visuelle.
2. **Événements privés** — anniversaires, baby showers, gender reveals, fêtes familiales.
3. **Événements professionnels** — séminaires, inaugurations, réceptions, lancements de produits.

Services proposés (4 axes)

1. Décoration personnalisée (couleurs, ambiance, tables, espaces)
2. Supports personnalisés (panneaux de bienvenue, panneaux directionnels, éléments graphiques)
3. Buffets et prestations culinaires (salé/sucré, mise en scène)
4. **Location de matériel** (tables, chaises, vaisselle, déco, accessoires) → justifie directement le module *catalogue* du site

Vision / mission

- Vision : devenir une **référence de l'événementiel haut de gamme** sur la Côte d'Azur.
- Mission : accompagnement complet du client, de la prise de contact à la réalisation finale.
- Valeurs : écoute, créativité, qualité, respect des engagements, attention aux détails.

Objectifs stratégiques (4)

- Développer la visibilité numérique

- Faciliter la relation client (contact rapide, devis, suivi)
- Valoriser les réalisations (galerie numérique)
- Optimiser l'organisation interne (admin, centralisation, suivi du matériel)

Constat actuel → besoin du projet

Gestion **non centralisée** aujourd'hui (échanges directs, canaux dispersés) → perte de visibilité et complexité de suivi → justifie la plateforme web.

CHAPITRE 3 : ÉTUDE DE L'EXISTANT ET SOLUTION PROPOSÉE

Objectif du chapitre

Analyser la situation actuelle de Maison Yara avant de concevoir la solution — identifier les méthodes de travail, les difficultés, et en déduire les besoins réels du projet.

1. Constat sur la présence numérique

- Pas de plateforme web complète.
- Communication limitée aux **réseaux sociaux + échanges directs**.
- Limite claire : impossible de structurer l'info, faciliter les demandes ou automatiser quoi que ce soit.

2. Fonctionnement actuel

Gestion 100% manuelle, canaux multiples et non centralisés : téléphone, réseaux sociaux, e-mail, rencontres physiques. Ça marche pour la relation client personnalisée, mais **ne tient pas la charge** si le volume augmente.

3. Gestion des demandes clients (devis)

Infos collectées manuellement (type d'événement, date, invités, lieu, besoins) → traitement manuel avec les limites classiques :

- pas de centralisation
- risque d'oubli
- pas de suivi automatique
- traitement lent
- pas d'historique consultable

4. Gestion des réalisations

Pas de galerie organisée → présentation peu pro, savoir-faire sous-exploité, expérience visiteur pauvre.

5. Gestion du matériel de location

Pas d'outil dédié → risques concrets : doublons de réservation, erreurs de disponibilité, inventaire flou.

6. Limites identifiées (synthèse en 4 points)

1. Manque de visibilité en ligne
2. Gestion manuelle des demandes

3. Absence de catalogue numérique
4. Manque de centralisation des données (clients/prestations/matériel dispersés)

7. Critique de l'existant

Le point positif à conserver : la relation directe/personnalisée avec les clients (atout dans un secteur basé sur la confiance). Le point bloquant : ce modèle **ne scale pas** avec la croissance de l'activité → besoin de digitaliser sans perdre cette proximité.

8. Solution proposée (rappel structurant)

Application en 2 blocs :

- **Partie publique** : présentation entreprise, services, galerie de réalisations, catalogue de location, formulaire de devis en ligne.
- **Partie admin** : auth sécurisée, gestion services/réalisations/devis/clients/catalogue/contenus/newsletter.

9. Bénéfices attendus

Visibilité ↑, gain de temps (automatisation), meilleure expérience client, organisation interne centralisée.

CHAPITRE 4 : ANALYSE DES BESOINS

Objectif du chapitre

Faire le pont entre l'étude de l'existant (constat des problèmes) et le cahier des charges technique (chapitre suivant) : traduire les besoins métier en fonctionnalités concrètes par acteur.

2 acteurs identifiés

1. Le Visiteur / Client (pas de compte requis)

- Consulter la présentation de l'entreprise (valeurs, activité, coordonnées)
- Consulter les services (titre + description + image)
- Consulter la galerie de réalisations (organisée par catégorie d'événement)
- Consulter le catalogue de location (image, nom, description, quantité dispo)
- Envoyer une demande de devis (nom, prénom, email, tél, description du besoin)
- S'inscrire à la newsletter

2. L'Administrateur (espace privé, authentifié)

- Authentification sécurisée : **JWT** (sessions) + **bcrypt** (mots de passe) + routes protégées
- Gestion des services (CRUD)
- Gestion des réalisations (CRUD + upload d'images)
- Gestion des demandes de devis, avec **statuts** : nouvelle → en cours → devis envoyé → accepté / refusé

- Gestion des clients (historique des échanges/projets)
- Gestion du catalogue de location (CRUD + disponibilité)
- Gestion de la newsletter (liste des abonnés + campagnes, via **Brevo**)

Besoins fonctionnels (synthèse en 5 blocs)

1. Contenu public (affichage entreprise/services/réalisations/catalogue)
2. Gestion des utilisateurs (auth, droits, protection des routes)
3. Demandes clients (envoi, enregistrement, consultation, statut)
4. Gestion des images (upload, stockage sécurisé, affichage optimisé, suppression)
5. Gestion du matériel (ajout, modification, quantités, disponibilités)

Besoins non fonctionnels

- **Performance** : images optimisées, requêtes limitées, code organisé
- **Sécurité** : auth, chiffrement, validation des saisies, contrôle des permissions
- **Ergonomie** : interface simple, moderne, intuitive
- **Compatibilité** : Chrome/Firefox/Edge/Safari + responsive (mobile/tablette/desktop)
- **SEO** : structure HTML optimisée, temps de chargement réduit, optimisation locale

User stories

4 côté visiteur (consulter services/galerie, envoyer devis, s'inscrire newsletter) + 4 côté admin (connexion, gérer services, gérer devis, gérer catalogue) — formulation classique "en tant que... je veux... afin de..."

CHAPITRE 5 : CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

Rôle du document

Formaliser noir sur blanc les besoins identifiés dans les chapitres précédents en objectifs, fonctionnalités, contraintes techniques — et servir de référence pour vérifier en fin de projet que la solution livrée correspond bien à ce qui était prévu.

Objectifs du projet (rappel synthétique)

Vitrine professionnelle + espace de gestion interne, avec 8 objectifs déclinés : visibilité, présentation claire des prestations, galerie dynamique, devis en ligne, communication facilitée, gestion du catalogue location, centralisation des infos, sécurisation des accès admin.

Solution = 2 blocs (déjà connu) : partie publique (vitrine + devis + newsletter) / partie admin (gestion complète, authentifiée).

7 modules fonctionnels détaillés

1. **Présentation entreprise** — infos modifiables par l'admin
2. **Gestion des services** — CRUD (id, titre, description, image, catégorie)
3. **Gestion des réalisations** — galerie, import sécurisé des images

4. **Demande de devis** — formulaire (nom, coordonnées, type d'événement, date, invités, description) → enregistré en base → visible côté admin
5. **Réservation** (*nouveau par rapport à avant*) — enregistrement + suivi des demandes, avec mention explicite d'une **évolution future vers un calendrier de disponibilité**
6. **Catalogue de location** — CRUD produits + gestion des quantités disponibles
7. **Newsletter** — abonnés + campagnes, via un service tiers (Brevo, déjà identifié dans les chapitres précédents)

Spécifications techniques

- **Architecture 3-tiers** : présentation / logique métier / données — c'est la première fois que cette architecture est nommée explicitement (à ajouter dans la partie technique du dossier).
- Front React, Back Node/Express, BDD MySQL — avec cette fois **la justification de chaque choix** (React → composants réutilisables ; Express → API REST légère ; MySQL → fiabilité, relationnel, compatibilité Node).

Sécurité (rien de nouveau mais formalisé)

JWT (sessions) + bcrypt (mots de passe) + validation des données en entrée.

Contraintes du projet (nouveau découpage en 3 catégories)

- *Techniques* : architecture moderne, multi-navigateurs, responsive, bonnes pratiques
- *Fonctionnelles* : simplicité, rapidité, facilité de gestion admin
- *Sécurité* : protection des données clients, contrôle des accès, sécurisation des fichiers importés

Planning prévisionnel (nouveau — 7 phases)

1. Analyse des besoins / étude de l'existant
2. Conception architecture + modélisation
3. Conception BDD
4. Développement Front-end
5. Développement Back-end
6. Intégration et tests
7. Déploiement + rédaction du mémoire

Ce qui est vraiment nouveau par rapport à ce qu'on a déjà dans le dossier Word :

- l'**architecture 3-tiers** nommée explicitement
- le **module Réservation** (distinct du devis, avec piste d'évolution "calendrier de disponibilité")
- le **planning prévisionnel en 7 phases**
- les contraintes classées en 3 catégories (technique / fonctionnelle / sécurité)

CHAPITRE 6 : CONCEPTION GÉNÉRALE DE LA SOLUTION

Voici une version reformulée et améliorée de ce chapitre — même structure et même contenu technique, mais avec une rédaction plus fluide, moins répétitive et davantage argumentée.

Chapitre : Conception générale de l'architecture

1. Introduction

L'analyse des besoins ayant permis de définir précisément les fonctionnalités attendues par Maison Yara by Berriri, il convient à présent de traduire ces exigences en une architecture technique cohérente. Cette étape de conception est déterminante : elle fixe l'organisation globale du système, la répartition des responsabilités entre ses composants et les choix technologiques qui conditionneront la suite du développement.

L'objectif est de concevoir une architecture à la fois moderne, sécurisée et évolutive, capable d'accompagner la croissance de l'entreprise sans nécessiter de refonte majeure à moyen terme. Le système repose ainsi sur une architecture **Client-Serveur en trois couches** :

- une **couche présentation**, correspondant à l'interface utilisateur ;
- une **couche métier**, chargée du traitement des données et de l'application des règles de gestion ;
- une **couche d'accès aux données**, responsable de la communication avec la base de données.

Cette séparation des responsabilités constitue le socle de la maintenabilité du projet : chaque couche peut évoluer indépendamment des autres, ce qui limite les effets de bord lors des futures évolutions.

2. Architecture générale du système

2.1. Une architecture Client-Serveur

Le **client** est représenté par l'application front-end développée avec React.js : c'est l'interface à travers laquelle les visiteurs et l'administratrice interagissent avec le système. Le **serveur** correspond à l'application back-end, développée avec Node.js et Express.js, qui traite les requêtes, applique les règles métier et orchestre les échanges avec la base de données. Enfin, **MySQL** constitue la couche de persistance où sont conservées durablement l'ensemble des informations du système.

2.2. Pourquoi ce choix d'architecture

Cette organisation présente plusieurs bénéfices concrets pour le projet :

- **Indépendance des couches** : le front-end et le back-end évoluent séparément — une modification de l'interface n'impose aucune modification du serveur, et inversement.
- **Maintenabilité** : un projet organisé en couches distinctes est plus simple à comprendre, à déboguer et à faire évoluer, y compris pour un développeur qui n'a pas participé à sa conception initiale.
- **Évolutivité** : cette base architecturale ouvre la voie à des extensions futures sans remise en cause du système existant — paiement en ligne, calendrier de disponibilité pour les réservations, application mobile dédiée, notifications automatiques, etc.
- **Réutilisabilité de l'API** : l'API REST développée côté serveur n'est pas liée à un client unique ; elle pourrait tout aussi bien alimenter une application mobile ou un autre front-end sans modification du back-end.

3. Détail des trois couches

3.1. Couche présentation (front-end)

Développée avec React.js, cette couche constitue l'unique point de contact entre les utilisateurs et le système. Elle a pour rôle d'afficher les informations, de capter les interactions utilisateur, de récupérer les données auprès de l'API et de transmettre les actions de l'utilisateur vers le serveur. Elle regroupe notamment les pages publiques, les composants réutilisables, les formulaires, ainsi que l'ensemble de l'espace d'administration.

3.2. Couche métier (back-end)

Développée avec Node.js et Express.js, cette couche constitue le cœur logique de l'application. Elle réceptionne les requêtes émises par le client, vérifie la validité des données reçues, applique les règles métier, communique avec MySQL et renvoie une réponse structurée au format JSON. Elle regroupe les routes de l'API, les contrôleurs, les services métier ainsi que les middlewares de sécurité (authentification, contrôle des rôles).

3.3. Couche données

Cette couche est assurée par la base de données relationnelle MySQL, qui centralise l'ensemble des informations du système : administrateurs, clients, services, réalisations, demandes de devis, réservations, produits du catalogue de location et abonnés à la newsletter. Elle garantit la persistance, la cohérence et la rapidité d'accès aux données, trois conditions indispensables au bon fonctionnement de la plateforme.

4. Organisation du back-end selon le modèle MVC

Pour structurer la partie serveur, le projet s'appuie sur une organisation inspirée du modèle **MVC (Model – View – Controller)**, qui isole clairement les responsabilités de chaque composant.

Le modèle correspond à la représentation des données manipulées par l'application — administrateur, client, service, réalisation, devis, réservation, produit, newsletter — et porte les opérations de création, modification, suppression et recherche associées à chaque entité.

Le contrôleur constitue la logique de traitement : il reçoit les requêtes en provenance du front-end, effectue les vérifications nécessaires, sollicite les fonctions métier appropriées, puis retourne une réponse au format JSON. Le déroulement d'une demande de devis illustre bien ce mécanisme : React transmet les informations saisies par le visiteur → le contrôleur reçoit la requête → les données sont contrôlées → la demande est enregistrée dans MySQL → une confirmation est renvoyée au client.

La vue, dans une architecture MVC traditionnelle, désigne l'interface utilisateur ; dans ce projet, ce rôle est intégralement pris en charge par React.js, qui gère l'affichage des pages, la navigation, les formulaires et l'ensemble des interactions.

5. Organisation du front-end

Le front-end React repose sur une **architecture à composants**, chaque composant représentant un fragment autonome et réutilisable de l'interface. Cette approche limite la duplication de code et facilite la cohérence visuelle du site. Parmi les composants structurants du projet :

- **Navbar** — gère la navigation entre les différentes pages du site ;
- **Footer** — regroupe les informations complémentaires (coordonnées, liens utiles, réseaux sociaux) ;
- **Gallery** — assure l'affichage dynamique des réalisations de l'entreprise ;
- **Newsletter** — gère le formulaire d'inscription des visiteurs aux actualités.

6. Organisation du back-end et exemples de routes API

Le serveur Express.js est structuré en modules distincts (routes, contrôleurs, modèles, middlewares), chacun correspondant à une responsabilité précise. Les routes définissent les points d'entrée exposés au front-end. Par exemple :

Route	Méthode	Usage
<code>/api/services</code>	GET	Récupérer la liste des services
<code>/api/services</code>	POST	Ajouter un nouveau service (administrateur)
<code>/api/login</code>	POST	Authentifier l'administrateur

Exemple de réponse pour `GET /api/services` :

json

```
[  
  
  {  
  
    "id_service": 1,  
  
    "title": "Décoration",  
  
    "description": "Service de décoration personnalisée"  
  
  }  
  
]
```

7. Communication front-end / back-end

Les échanges entre React et Express reposent sur une **API REST**, fondée sur le protocole HTTP et le format d'échange **JSON**, léger et facile à manipuler côté client comme côté serveur.

Méthode HTTP	Usage
GET	Récupérer des données
POST	Ajouter des données
PUT	Modifier des données
DELETE	Supprimer des données

8. Justification des choix technologiques

Technologie	Rôle dans le projet	Justification du choix
-------------	---------------------	------------------------

React.js	Interface utilisateur	Interfaces dynamiques, composants réutilisables, large communauté, maintenabilité
Node.js / Express.js	API REST	Développement rapide, gestion efficace des requêtes HTTP, architecture flexible
MySQL	Base de données	Stabilité, modèle relationnel adapté aux données du projet, large compatibilité
Multer	Upload de fichiers	Réception et stockage des images depuis l'espace d'administration
JWT	Authentification	Identification sécurisée, protection des routes privées, gestion des sessions
bcrypt	Sécurité des mots de passe	Hachage des mots de passe avant tout enregistrement en base
Brevo	Newsletter	Envoi automatisé des campagnes, gestion des abonnés, meilleure délivrabilité
Swiper.js	Carrousels	Présentation fluide et attractive des réalisations, adaptée au mobile

9. Conclusion

Cette phase de conception a permis de fixer l'organisation globale de l'application et les interactions entre ses différents composants. L'architecture retenue — séparation claire entre interface, logique métier et gestion des données, complétée par une structuration MVC côté serveur — garantit à la fois une maintenabilité durable, une sécurité renforcée et une capacité d'évolution alignée sur les ambitions de développement de Maison Yara.

CHAPITRE 7 : CONCEPTION DÉTAILLÉE (MERISE ET UML)

1. Introduction

La définition de l'architecture générale a fixé le cadre global de la solution ; il s'agit à présent d'entrer dans le détail de la conception, afin de représenter précisément les données manipulées, les fonctionnalités attendues et les interactions entre les différents acteurs du système. Cette étape transforme les besoins fonctionnels, déjà identifiés, en modèles techniques directement exploitables lors du développement.

Deux approches complémentaires structurent cette conception :

- la méthode **MERISE**, pour modéliser les données et concevoir la base de données ;
- le langage **UML** (*Unified Modeling Language*), pour représenter les fonctionnalités, les comportements et la structure interne de l'application.

Croiser ces deux approches permet d'obtenir une vision complète du système — à la fois du point de vue des données qu'il conserve et des traitements qu'il exécute.

2. La méthode MERISE

MERISE est une méthode de conception structurée des systèmes d'information, qui distingue clairement les **données** manipulées par le système des **traitements** qui les exploitent. Elle vise à produire une base de données cohérente, normalisée et alignée sur les besoins fonctionnels du projet, à travers trois niveaux de représentation successifs : le **Modèle Conceptuel de Données (MCD)**, le **Modèle Logique de Données (MLD)** et le **Modèle Physique de Données (MPD)**. Ce chapitre couvre les deux premiers niveaux ; le MPD (types SQL précis, index, contraintes) est traité dans la partie « Mise en place de la base de données » du dossier.

3. Modèle Conceptuel de Données (MCD)

Le MCD représente la structure des informations manipulées par l'application, indépendamment de tout système de gestion de base de données. Il identifie les entités du système, leurs attributs, et les relations qui les unissent — une représentation abstraite, préalable à toute transformation en tables relationnelles.

4. Les entités du système

L'analyse des besoins de Maison Yara a permis d'identifier huit entités principales :

Entité	Rôle
Administrateur	Gestion de la plateforme, authentification

Service	Prestation proposée par l'entreprise
Réalisation	Événement passé, alimentant la galerie publique
Devis	Demande de prestation envoyée par un client
Réservation	Réservation liée à un événement (matériel et/ou prestation)
Produit	Élément du catalogue de location
Newsletter	Abonné aux actualités de l'entreprise

Les modules suivants ont été **effectivement implémentés** dans la version livrée de l'application :

Module de présentation de l'entreprise

Affichage des informations relatives à Maison Yara (présentation, valeurs, zones d'intervention, contact).

Module de gestion des services

Table **services** — chaque service comprend un titre, une description et une image. L'administrateur dispose des opérations de création, modification, suppression et consultation.

Module de gestion des réalisations

Table **realisations** — chaque réalisation comprend un titre, une description et une image, alimentant la galerie publique. Import des images géré par Multer.

Module de demande de devis

Table **devis** — le visiteur transmet nom, coordonnées, type d'événement, date souhaitée, nombre d'invités et description du besoin. Chaque demande est enregistrée avec un statut de suivi (**En attente** par défaut).

Module de newsletter

Table **newsletter** — inscription des visiteurs par e-mail (contrainte d'unicité en base), consultable par l'administrateur.

Modules identifiés comme besoins mais non implémentés dans cette version

Deux fonctionnalités figuraient dans l'expression de besoin initiale de la cliente et dans le cahier des charges fonctionnel, mais n'ont pas été intégrées à la base de données de la version livrée :

- **Catalogue de location** (gestion du matériel événementiel — tables, chaises, vaisselle, etc.)
- **Module de réservation** (suivi des réservations liées à un événement)

Ces deux modules restent des **perspectives d'évolution** du projet (voir section « Limites et perspectives »), et non des fonctionnalités livrées. Leur absence n'invalide pas l'analyse des besoins réalisée en amont — elle reflète simplement un périmètre de développement resserré sur les fonctionnalités jugées prioritaires pour une première version opérationnelle.

2. Conception de l'architecture — couche données (correction)

À remplacer dans le chapitre « Conception générale de l'architecture », section « Couche données ».

Cette couche correspond à la base de données MySQL **maison_yara**. Elle stocke cinq entités, sans lien de clé étrangère entre elles — chaque table fonctionne de manière autonome :

- **admin** — comptes du back-office ;
- **services** — prestations affichées sur le site ;
- **realisations** — portfolio des événements réalisés ;
- **devis** — demandes envoyées via le formulaire de contact ;
- **newsletter** — abonnés aux actualités.

(Les entités Client, Produit et Réservation, envisagées lors de l'analyse des besoins, ne font pas partie du schéma de cette version — voir « Limites et perspectives ».)

Modèle (MVC) — correction

Le modèle manipule les entités **Administrateur**, **Service**, **Réalisation**, **Devis** et **Newsletter**. Chacune correspond à une table indépendante ; il n'existe pas de relations entre elles dans la version actuelle de la base.

3. Conception détaillée — MCD / MLD / MPD

3.1. Une conception simplifiée par rapport à l'analyse initiale

L'analyse des besoins avait fait émerger des entités potentielles telles que Client, Produit (catalogue de location) et Réservation. Au moment de la conception détaillée et de l'implémentation, le périmètre a été resserré autour des fonctionnalités jugées prioritaires pour une première version opérationnelle : présentation de l'entreprise, gestion des services et réalisations, demandes de devis, newsletter. Les modules catalogue et réservation ont été conservés comme perspectives d'évolution plutôt que développés dans cette itération.

3.2. Modèle Conceptuel de Données (MCD)

Le MCD retenu comprend **cinq entités indépendantes**, sans association entre elles :

Entité	Attributs principaux
ADMINISTRATEUR	id, email, mot de passe
SERVICE	id, titre, description, image, catégorie, date de création
REALISATION	id, titre, description, image, lieu, date de l'événement, date de création
DEVIS	id, nom complet, email, téléphone, type d'événement, message, nombre d'invités, date de l'événement, statut, date de création
ABONNE_NEWSLETTER	id, email, date d'inscription

Le modèle réellement implémenté ne comporte **aucune relation formalisée en base**. Chaque demande de devis porte directement les coordonnées du client (nom, e-mail, téléphone) sans passer par une entité Client séparée.

3.3. Modèle Logique de Données (MLD)

Chaque entité devient une table ; l'absence d'association se traduit par l'absence de clé étrangère.

Table	Clé primaire	Contrainte particulière
admin	id	—
services	id_service	—
realisations	id_realisation	—
devis	id_devis	—
newsletter	id	email unique

3.4. Modèle Physique de Données (MPD)

sql

```
CREATE TABLE admin (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  email VARCHAR(255) NOT NULL,  
  password VARCHAR(255) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
CREATE TABLE services (  
  id_service INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,
```

```
image VARCHAR(255),  
category VARCHAR(100),  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE realisations (  
id_realisation INT AUTO_INCREMENT PRIMARY KEY,  
title VARCHAR(255) NOT NULL,  
description TEXT,  
image VARCHAR(255),  
location VARCHAR(255),  
event_date DATE,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE devis (  
id_devis INT AUTO_INCREMENT PRIMARY KEY,  
fullname VARCHAR(255),  
email VARCHAR(255),  
phone VARCHAR(50),  
event_type VARCHAR(255),  
message TEXT,  
event_date DATE,  
nombre_invites INT,  
statut VARCHAR(20) DEFAULT 'En attente',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE newsletter (

  id INT AUTO_INCREMENT PRIMARY KEY,

  email VARCHAR(255) NOT NULL UNIQUE,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3.5. Diagramme de cas d'utilisation

Le visiteur peut : consulter les services, consulter la galerie de réalisations, envoyer une demande de devis, s'inscrire à la newsletter.

(Retrait de « consulter le catalogue de location », non implémenté.)

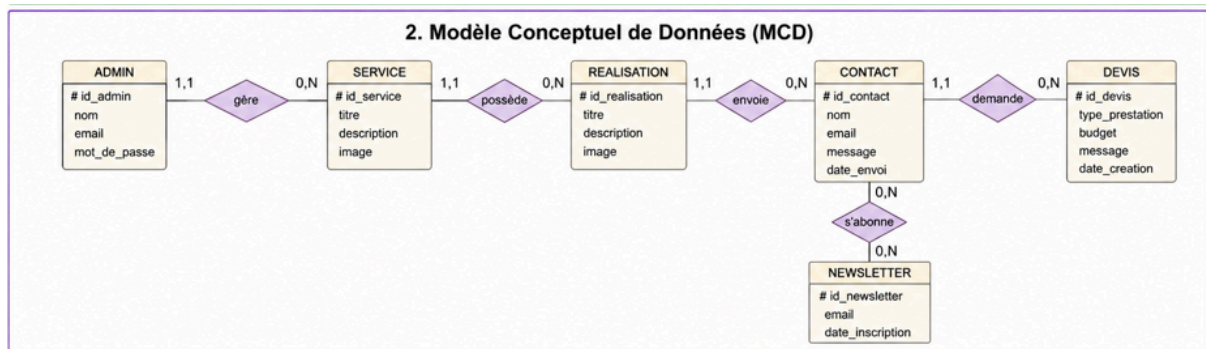
L'administrateur peut : se connecter, gérer les services, gérer les réalisations, consulter et suivre les devis, consulter les abonnés à la newsletter.

4. Limites et perspectives d'évolution

La version livrée de l'application couvre l'ensemble des fonctionnalités jugées prioritaires : vitrine de l'entreprise, présentation des services, galerie de réalisations, demandes de devis et newsletter. Deux besoins identifiés lors de l'analyse initiale n'ont pas été intégrés à cette version :

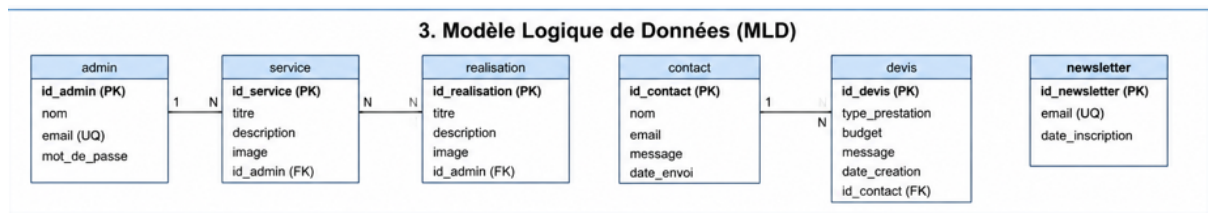
- **Le catalogue de location de matériel événementiel**, qui nécessiterait l'ajout d'une entité Produit avec gestion des stocks et des disponibilités ;
- **Le suivi des réservations**, qui nécessiterait une entité Reservation reliée aux services et, le cas échéant, aux produits du catalogue.

L'architecture actuelle — base MySQL simple, API REST modulaire — permettrait d'intégrer ces évolutions sans remise en cause du système existant : il suffirait d'ajouter les tables correspondantes et les routes API associées, sans modifier les modules déjà en production.



6. Modèle Logique de Données (MLD)

Le MLD traduit le modèle conceptuel en une structure exploitable par un système relationnel : chaque entité devient une table, chaque relation devient une clé étrangère — ou, dans le cas d'une relation plusieurs-à-plusieurs, une table associative dédiée.



7. Modélisation UML

UML est un langage graphique modélisation permettant de représenter les utilisateurs du système, ses fonctionnalités, ses interactions et sa structure interne. Quatre diagrammes sont mobilisés dans ce projet : cas d'utilisation, classes, séquence et activité.

8. Diagramme de cas d'utilisation

Ce diagramme identifie les interactions entre les acteurs et le système. Deux acteurs principaux interviennent :

Le visiteur peut consulter les services, la galerie et le catalogue, envoyer une demande de devis, contacter l'entreprise et s'inscrire à la newsletter.

L'administrateur peut se connecter, gérer les services, les réalisations et les produits, consulter les devis et gérer les abonnés à la newsletter.

9. Diagramme de classes

10. Diagrammes de séquence

Authentification administrateur : l'administrateur saisit ses identifiants → React transmet les informations à l'API → le serveur vérifie les données dans MySQL → un token JWT est généré → l'accès au tableau de bord est autorisé.

Demande de devis : le client remplit le formulaire → les données sont envoyées au serveur → le contrôleur vérifie les informations → la demande est enregistrée dans MySQL → une confirmation est retournée au client.

11. Diagrammes d'activité

Connexion administrateur : ouverture de la page de connexion → saisie des identifiants → vérification des informations → génération du token → accès au tableau de bord.

Ajout d'une réalisation : ouverture de l'espace de gestion → sélection de l'ajout d'une réalisation → saisie des informations et de l'image → enregistrement des données par le serveur → apparition de la réalisation dans la galerie publique.

12. Conclusion

Cette phase de conception détaillée a permis de représenter précisément la structure et le fonctionnement de l'application : MERISE a fixé l'organisation des données et la structure de la base MySQL, tandis qu'UML a modélisé les interactions entre les utilisateurs et le système. Cette base solide prépare directement l'étape suivante : la réalisation de l'application, les choix d'implémentation et la présentation des interfaces développées.

CHAPITRE 8 : RÉALISATION DE L'APPLICATION

1. Introduction

À la suite des phases d'analyse et de conception, le développement de l'application a permis de concrétiser les besoins exprimés par Maison Yara by Berriri en une solution web complète. L'objectif était de mettre en place une plateforme moderne permettant à la fois de présenter les prestations de l'entreprise, de faciliter les échanges avec les clients et de simplifier les tâches de gestion réalisées par l'administrateur.

La réalisation s'est appuyée sur les technologies retenues lors de la phase de conception — React.js pour le front-end, Node.js et Express.js pour le back-end, MySQL pour la base de données — et a été organisée selon une approche progressive : chaque fonctionnalité a été développée, testée puis intégrée, afin de garantir la cohérence de l'ensemble de l'application.

Ce chapitre présente l'environnement de développement utilisé, l'organisation du code, les principales fonctionnalités implémentées ainsi que les interfaces développées.

2. Environnement de développement

Environnement matériel

Élément	Description
Processeur	AMD Ryzen 7 3700U with Radeon Vega Mobile Gfx (2,30 GHz)
Mémoire RAM	16 Go (13,9 Go utilisables)
Carte graphique	AMD Radeon(TM) RX Vega 10 Graphics (2 Go)
Stockage	477 Go (190 Go utilisés)
Système d'exploitation	

Environnement logiciel

- **Visual Studio Code** : environnement principal de développement (écriture du code, extensions, débogage, organisation des fichiers).
- **Node.js** : environnement d'exécution du serveur.
- **npm** : gestion des dépendances (React Router, Express, mysql2, bcrypt, jsonwebtoken, Multer...).
- **MySQL** : création des tables, gestion des relations, exécution des requêtes SQL.
- **Git** : suivi des modifications et historique du développement.

3. Organisation générale du projet

L'application est organisée en deux parties indépendantes — le front-end et le back-end — chacune disposant de son propre `package.json` et de son propre cycle de développement. Cette séparation facilite l'organisation du code et l'évolution future de la plateforme.

(À compléter avec les arborescences réelles des dossiers `front/` et `back/`.)

4. Réalisation de la partie publique

La partie publique constitue l'interface accessible à tous les visiteurs, sans authentification. Elle permet de découvrir l'univers de Maison Yara, de consulter les prestations proposées et d'effectuer différentes actions. L'interface a été conçue dans un souci d'ergonomie, de simplicité de navigation et de mise en valeur des réalisations de l'entreprise.

4.1. Page d'accueil

Point d'entrée principal de l'application, la page d'accueil présente une vue d'ensemble de l'activité de Maison Yara à travers une mise en page dynamique mettant en avant les principaux services. Elle comprend une bannière de présentation, une introduction à l'entreprise, un aperçu des prestations ainsi que des liens permettant d'accéder rapidement aux différentes rubriques du site. L'objectif est de capter l'attention du visiteur et de lui offrir un accès rapide aux informations essentielles.

4.2. Présentation des services

Chaque prestation est présentée sous forme de carte contenant une image, un titre et une description. Les informations affichées sont récupérées dynamiquement depuis l'API REST, ce qui permet à l'administrateur de mettre à jour le contenu sans modifier le code de l'application — une gestion flexible qui facilite l'évolution des prestations proposées.

4.3. Galerie des réalisations

La galerie valorise les événements organisés par Maison Yara à travers une sélection de photographies, affichées de manière dynamique et présentées sous forme de carrousel grâce à l'intégration de **Swiper.js**, garantissant une navigation fluide sur desktop comme sur mobile. Cette section joue un rôle stratégique dans la communication de l'entreprise, puisqu'elle met en évidence son savoir-faire et la qualité de ses prestations.

4.4. Catalogue de location

Le catalogue présente les équipements disponibles à la location, chacun accompagné d'une illustration et d'une description permettant aux visiteurs d'obtenir rapidement les informations nécessaires. Cette fonctionnalité complète naturellement les prestations d'organisation d'événements.

4.5. Formulaire de demande de devis

Fonctionnalité centrale de l'application, ce formulaire permet au visiteur de renseigner les informations relatives à son projet : type d'événement, date souhaitée, nombre d'invités et description de ses besoins. Après validation, les informations sont transmises au serveur puis enregistrées en base de données afin d'être traitées par l'administrateur — réduisant les échanges manuels et améliorant le suivi des demandes.

4.6. Page de contact

Cette page permet aux visiteurs de transmettre directement un message à l'entreprise et regroupe les principales informations de contact, contribuant à renforcer la proximité avec les clients.

5. Réalisation de l'espace d'administration

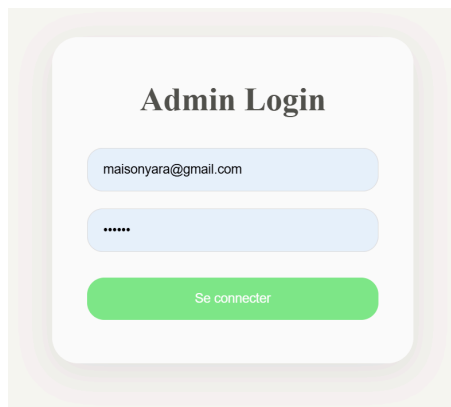
L'espace d'administration constitue la partie privée de l'application, dont l'accès est réservé aux utilisateurs autorisés grâce à un système d'authentification sécurisé. Il permet de gérer l'ensemble des contenus affichés sur le site public ainsi que les informations transmises par les visiteurs.

5.1. Authentification

Avant d'accéder aux fonctionnalités d'administration, l'utilisateur saisit son adresse électronique et son mot de passe. Le mécanisme de vérification se déroule comme suit :

1. L'administrateur saisit ses identifiants.
2. Le serveur recherche l'administrateur correspondant dans la base de données.
3. Le mot de passe saisi est comparé au mot de passe haché grâce à **bcrypt**.
4. Si la vérification réussit, un token **JWT** est généré et renvoyé au client ; en cas d'identifiants invalides, l'accès est refusé.
5. Ce token est ensuite transmis à chaque requête protégée pour autoriser l'accès aux fonctionnalités d'administration.

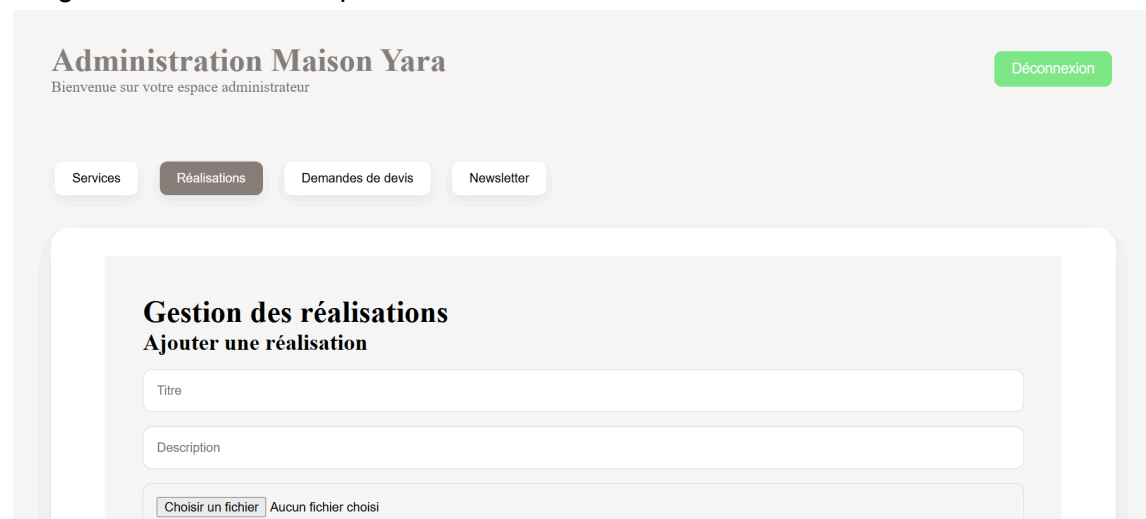
Ce mécanisme garantit que seules les personnes autorisées peuvent modifier les contenus de l'application.



The image shows a login form titled "Admin Login". It features two input fields: the first contains the email address "maisonyara@gmail.com", and the second contains masked characters "*****". Below these fields is a green button labeled "Se connecter". The form is set against a light beige background with a subtle shadow effect.

5.2. Tableau de bord

Le tableau de bord centralise les différentes fonctionnalités de gestion et constitue le point de départ des opérations d'administration : nombre de demandes reçues, de services, de réalisations et d'abonnés à la newsletter. L'interface a été conçue pour simplifier la navigation et améliorer la productivité de l'administratrice.

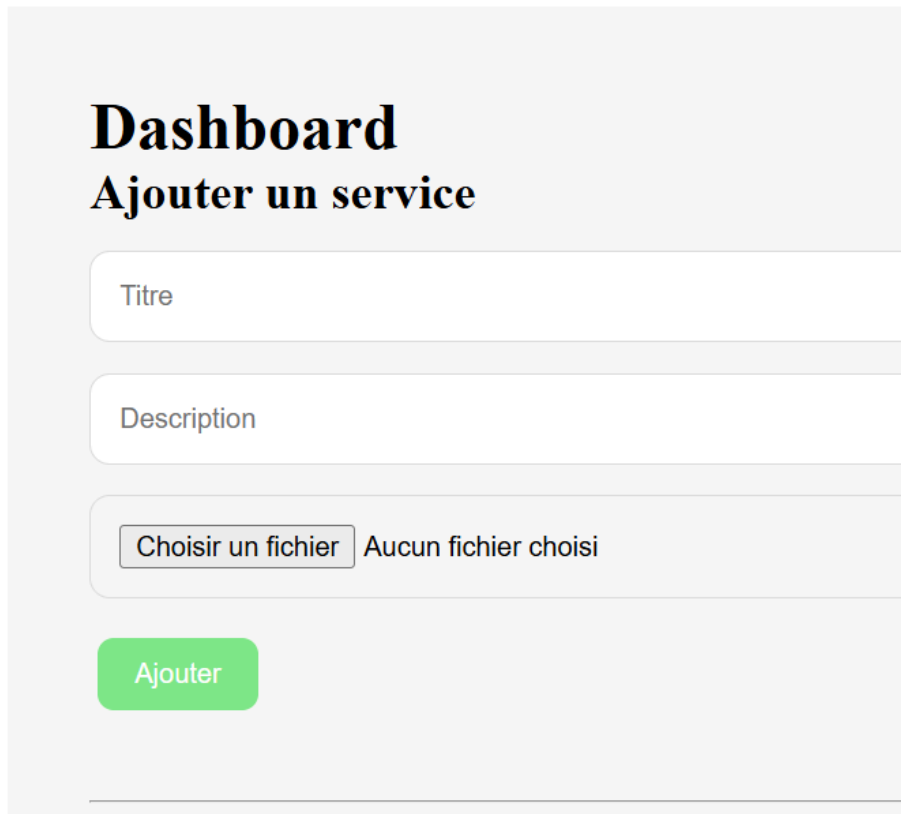


The image displays the "Administration Maison Yara" dashboard. At the top, the title "Administration Maison Yara" is followed by the subtitle "Bienvenue sur votre espace administrateur". A green "Déconnexion" button is located in the top right corner. Below the header, there is a navigation bar with four buttons: "Services", "Réalisations" (which is highlighted), "Demandes de devis", and "Newsletter". The main content area is titled "Gestion des réalisations" and includes a sub-header "Ajouter une réalisation". This section contains three input fields: "Titre", "Description", and a file upload field labeled "Choisir un fichier" with the text "Aucun fichier choisi" next to it.

5.3. Gestion des services

L'application permet d'ajouter, de modifier et de supprimer les prestations proposées ; chaque modification est immédiatement répercutée sur la partie publique du site, ce qui

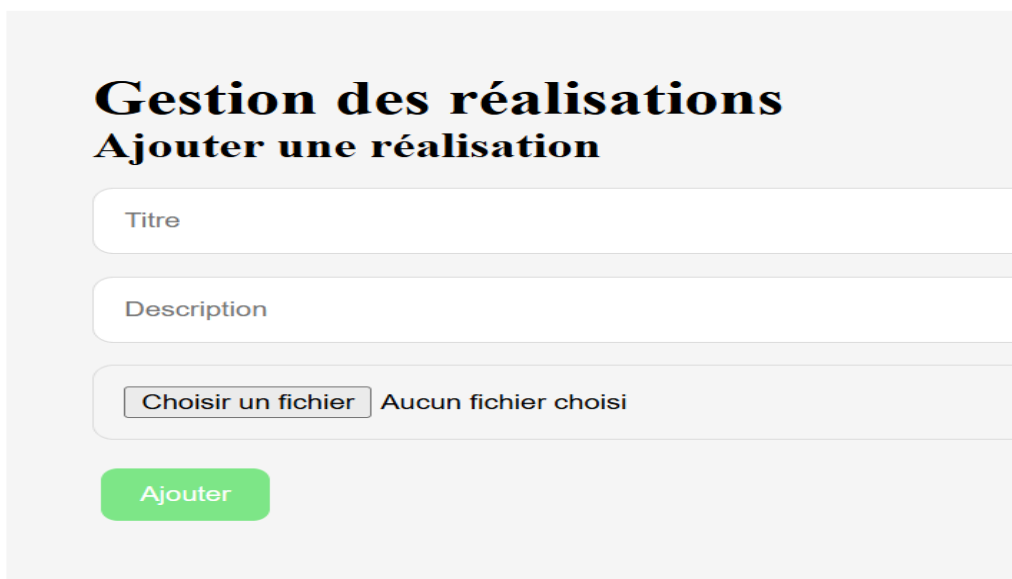
permet de maintenir le contenu constamment à jour.



The screenshot shows a web form titled "Ajouter un service" under a "Dashboard" header. The form consists of three main sections: a "Titre" (Title) input field, a "Description" input field, and a file selection section. The file selection section includes a button labeled "Choisir un fichier" and a text label "Aucun fichier choisi". At the bottom of the form is a green "Ajouter" (Add) button.

5.4. Gestion des réalisations

L'administrateur peut enrichir la galerie en ajoutant de nouvelles réalisations ou en supprimant celles qui ne sont plus d'actualité. L'import d'image est géré par **Multer** selon le processus suivant : sélection de l'image par l'administrateur → envoi du fichier au serveur → traitement de l'importation → stockage dans le dossier prévu → enregistrement du chemin en base de données.



The screenshot shows a web form titled "Ajouter une réalisation" under a "Gestion des réalisations" header. The form structure is identical to the one above, with fields for "Titre", "Description", a file selection button "Choisir un fichier", a text label "Aucun fichier choisi", and a green "Ajouter" button at the bottom.

5.5. Gestion des demandes de devis

Les demandes envoyées par les visiteurs sont centralisées dans une interface dédiée : l'administrateur peut consulter les informations transmises, suivre les nouvelles demandes et assurer leur traitement — une centralisation qui améliore l'organisation interne et réduit les risques d'oubli.

(Insérer capture d'écran — Figure 8.12 : Gestion des demandes de devis)

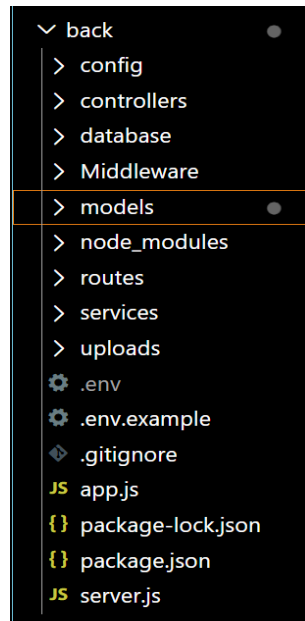
6. Création de l'API REST

L'application communique avec le front-end grâce à une API REST. Les principales routes développées sont les suivantes :

Méthode	Route	Fonction
GET	<code>/api/services</code>	Afficher les services
POST	<code>/api/services</code>	Ajouter un service (<i>admin</i>)
PUT	<code>/api/services/:id</code>	Modifier un service (<i>admin</i>)
DELETE	<code>/api/services/:id</code>	Supprimer un service (<i>admin</i>)
GET	<code>/api/realisations</code>	Afficher les réalisations
POST	<code>/api/realisations</code>	Ajouter une réalisation (<i>admin</i>)
GET	<code>/api/catalogue</code>	Afficher le catalogue de location
POST	<code>/api/devis</code>	Envoyer une demande de devis
GET	<code>/api/devis</code>	Consulter les demandes (<i>admin</i>)
POST	<code>/api/newsletter/subscriber</code>	S'inscrire à la newsletter
POST	<code>/api/login</code>	Authentifier l'administrateur

7. Gestion des données

Toutes les informations manipulées par l'application — services, réalisations, demandes de devis, abonnements — sont enregistrées dans la base de données relationnelle MySQL, de manière centralisée afin d'assurer leur cohérence et leur disponibilité. Les échanges entre



l'application et la base de données sont réalisés automatiquement à travers les différentes routes de l'API.

8. Sécurisation de l'application

Plusieurs mécanismes renforcent la sécurité de la plateforme :

- l'authentification des administrateurs repose sur des jetons **JWT**, protégeant les pages privées ;
- les mots de passe sont chiffrés avec **bcrypt** avant leur enregistrement, assurant la confidentialité des informations sensibles ;
- les données saisies dans les formulaires sont contrôlées avant leur traitement, afin de limiter les erreurs et les tentatives d'injection de données malveillantes.

9. Responsive design

L'application a été conçue selon une approche responsive, garantissant une adaptation automatique aux différentes tailles d'écran (ordinateur, tablette, smartphone) — une exigence incontournable au regard de la part croissante de consultation depuis des appareils mobiles.

(Insérer captures mobile/tablette — Figure 8.14 : Adaptation responsive de l'application)

10. Bilan de la réalisation

La réalisation de cette application a permis de développer une plateforme web répondant aux principaux besoins de Maison Yara by Berriri. Les fonctionnalités développées facilitent aussi bien la consultation des services par les visiteurs que la gestion quotidienne des contenus par l'administrateur. Grâce à son architecture modulaire et à son interface responsive, l'application peut évoluer facilement pour intégrer de nouvelles fonctionnalités selon les futurs besoins de l'entreprise.

11. Conclusion

La phase de réalisation a permis de concrétiser les choix effectués lors des étapes d'analyse et de conception en une application web complète et fonctionnelle. Les différentes interfaces développées répondent aux objectifs fixés au début du projet et offrent une solution moderne, adaptée aux activités de Maison Yara by Berriri.

CHAPITRE 9 : TESTS, VALIDATION ET DÉPLOIEMENT DE L'APPLICATION

1. Introduction

Une fois le développement de l'application achevé, une phase de tests rigoureuse est indispensable pour vérifier son bon fonctionnement et s'assurer que les fonctionnalités développées répondent effectivement aux besoins exprimés par Maison Yara by Berriri. Cette étape permet de détecter les anomalies avant mise en production, de valider chaque module de l'application et de garantir une expérience utilisateur satisfaisante — c'est également un prérequis incontournable avant le déploiement de la solution.

Ce chapitre présente les tests réalisés, les résultats obtenus, la procédure de déploiement ainsi que les principales difficultés rencontrées durant le développement.

2. Tests fonctionnels — partie publique

Chaque module a été testé individuellement avant son intégration au système complet, en couvrant à la fois les scénarios attendus et les cas d'erreur.

N°	Fonctionnalité testée	Éléments vérifiés	Résultat
T-01	Page d'accueil	Menu de navigation, carrousel principal, section services, pied de page	Succès — tous les composants se chargent sans erreur
T-02	Page des services	Récupération des données API, affichage des cartes, chargement des images et descriptions	Succès — contenu conforme aux données enregistrées en base
T-03	Galerie des réalisations	Chargement des images, fonctionnement du carrousel (Swiper.js), affichage des réalisations	Succès — navigation fluide, aucune image manquante
T-04	Formulaire de devis (cas valide)	Saisie complète, validation des champs obligatoires, envoi au serveur, enregistrement en base	Succès — demande visible depuis l'espace administrateur

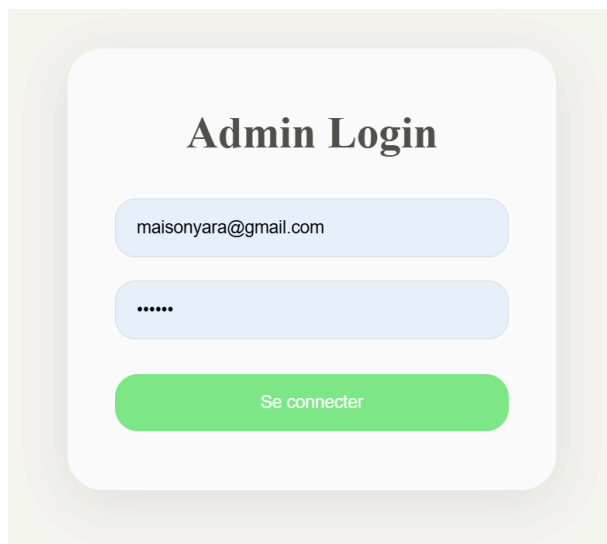
T-05	Formulaire de devis (champ manquant)	Soumission avec un champ obligatoire vide (ex. email)	Succès — formulaire bloqué, message d'erreur affiché, aucune requête envoyée au serveur
T-06	Page de contact	Saisie des informations, validation des champs, envoi du message	Succès — message transmis et réceptionné côté serveur

(Figures 9.1 à 9.5 : captures d'écran de la page d'accueil, des services, de la galerie, du formulaire de devis et de la page de contact — à insérer.)

3. Validation de l'espace administrateur

3.1. Authentification

N°	Scénario	Données	Résultat attendu	Résultat obtenu
T-07	Connexion avec identifiants valides	email + mot de passe corrects	Redirection vers le tableau de bord	Succès
T-08	Connexion avec mot de passe incorrect	email valide, mot de passe erroné	Message d'erreur générique, accès refusé	Succès
T-09	Accès direct au tableau de bord sans connexion	URL /admin/dashboard sans token	Redirection vers la page de connexion	Succès



3.2. Tableau de bord

Après authentification, l'administrateur accède au tableau de bord, qui centralise les fonctionnalités de gestion : services, réalisations, demandes de devis et administration des

contenus. La navigation entre ces modules a été testée et validée sans erreur de chargement.

3.3. Gestion des services (

The screenshot shows the 'Administration Maison Yara' interface. At the top, there's a header with the title 'Administration Maison Yara' and a subtitle 'Bienvenue sur votre espace administrateur'. A green 'Déconnexion' button is in the top right. Below the header, there are four navigation buttons: 'Services', 'Réalisations' (which is highlighted), 'Demandes de devis', and 'Newsletter'. The main content area is titled 'Gestion des réalisations' with a subtitle 'Ajouter une réalisation'. It contains three input fields: 'Titre', 'Description', and a file upload section with a button 'Choisir un fichier' and the text 'Aucun fichier choisi'.

CRUD)

N°	Opération	Résultat
T-1 0	Ajout d'un service	Succès — le service apparaît immédiatement sur le site public
T-1 1	Modification d'un service	Succès — les changements sont répercutés sans délai
T-1 2	Suppression d'un service	Succès — le service disparaît de la page publique
T-1 3	Affichage de la liste des services	Succès — liste conforme au contenu de la base de données

3.4. Gestion des réalisations

L'ajout et la suppression de réalisations ont été testés, y compris l'import d'image via **Multer** : les fichiers sont correctement stockés dans le dossier **uploads/** et apparaissent automatiquement dans la galerie publique après validation.

N°	Opération	Résultat
T-1 4	Ajout d'une réalisation avec image valide (< 5 Mo, JPEG/PNG)	Succès — image stockée et visible dans la galerie

T-1 5	Ajout d'une image trop volumineuse (> 5 Mo)	Succès — upload refusé, message d'erreur affiché
T-1 6	Suppression d'une réalisation	Succès — réalisation et image associée supprimées

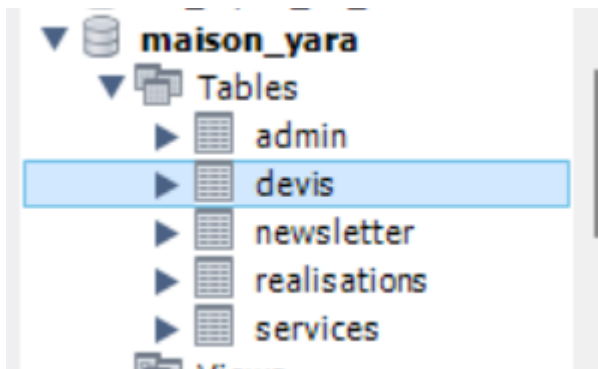
4. Tests de l'API REST

Les différentes routes de l'API ont été testées avec **Postman** afin de vérifier la communication entre React et Express, ainsi que la cohérence des réponses renvoyées.

Méthod e	Route	Cas testé	Résultat
GET	<code>/api/services</code>	Récupération de la liste des services	Succès (200)
POST	<code>/api/services</code>	Ajout d'un service (avec token admin valide)	Succès (201)
POST	<code>/api/services</code>	Ajout d'un service sans token	Échec attendu (401)
PUT	<code>/api/services/:id</code>	Modification d'un service existant	Succès (200)
DELETE	<code>/api/services/:id</code>	Suppression d'un service existant	Succès (200)
POST	<code>/api/devis</code>	Envoi d'une demande de devis valide	Succès (201)
POST	<code>/api/login</code>	Connexion avec identifiants incorrects	Échec attendu (401)

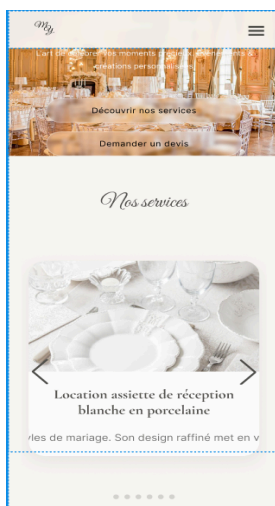
5. Validation de la base de données

Les données enregistrées via les différents formulaires ont été vérifiées directement dans MySQL, table par table : `admin`, `services`, `realisations`, `devis`, `newsletter`. Les enregistrements sont correctement sauvegardés, cohérents avec les données saisies côté front, et récupérables sans perte d'information par l'application.



6. Tests responsive

L'application a été testée sur trois types de supports (ordinateur, tablette, smartphone), en contrôlant l'adaptation des menus, le redimensionnement des images, l'affichage des formulaires et la fluidité de la navigation. Les résultats confirment que l'application reste pleinement utilisable quelle que soit la taille de l'écran.



7. Déploiement de l'application

Une fois les tests validés, l'application peut être préparée pour son déploiement en production, selon les étapes suivantes :

1. **Build du front-end** : `npm run build` génère une version React optimisée (fichiers minifiés, assets compressés) destinée à la production.
2. **Configuration du back-end** : les variables d'environnement de production sont définies (connexion à la base de données, `JWT_SECRET`, clé API Brevo), sans jamais versionner ces valeurs dans Git.
3. **Mise en place de la base de données** : la base MySQL est créée sur le serveur de production, puis les migrations (et éventuellement les seeders) sont exécutées pour reproduire la structure validée en développement.

4. **Mise en ligne** : le back-end est démarré sur le serveur (ou une plateforme d'hébergement type VPS/PaaS), le front-end compilé est servi soit par le même serveur, soit via un hébergement statique dédié.
5. **Vérifications post-déploiement** : test des principales routes en production, vérification du HTTPS et du bon fonctionnement de l'authentification.

Cette organisation en couches distinctes facilite le déploiement puisque chaque partie (front, back, base de données) peut être mise à jour indépendamment.

Le déploiement définitif sera réalisé à l'issue de la validation finale du projet.

8. Difficultés rencontrées

Trois difficultés techniques principales ont marqué le développement :

- **Communication front-end / back-end** : la mise en place initiale des échanges entre React et Express a nécessité une configuration précise des routes de l'API et de la gestion des requêtes HTTP (CORS notamment) — résolue par une organisation claire des routes et un typage cohérent des réponses JSON.
- **Gestion des fichiers images** : l'upload depuis l'espace administrateur a été fiabilisé grâce à **Multer**, qui assure un stockage et un affichage cohérents des fichiers importés.
- **Sécurisation de l'espace administrateur** : la mise en place de l'authentification **JWT**, couplée au chiffrement des mots de passe avec **bcrypt**, a permis d'obtenir un mécanisme d'accès fiable et sécurisé.

Chacune de ces difficultés a été surmontée par une approche progressive : développement, test isolé, puis intégration au système complet.

9. Conclusion

Les tests réalisés ont permis de valider le bon fonctionnement de l'application développée pour Maison Yara by Berriri. Les fonctionnalités principales — gestion des services, des réalisations, des demandes de devis et de l'espace d'administration — répondent aux besoins exprimés lors de la phase d'analyse. L'application constitue ainsi une solution moderne, performante et évolutive, prête à accompagner le développement numérique de l'entreprise.

Conclusion générale

Ce projet a permis de concevoir et de développer une application web complète pour Maison Yara by Berriri, entreprise spécialisée dans l'organisation d'événements privés et professionnels sur la Côte d'Azur. Il s'inscrit dans une problématique clairement identifiée dès le début de la démarche : comment permettre à une entreprise événementielle de renforcer sa présence numérique, de faciliter sa relation avec ses clients et d'optimiser la

gestion de ses activités, alors que son fonctionnement reposait jusqu'ici sur des échanges dispersés (téléphone, réseaux sociaux, rencontres physiques) et une gestion manuelle du matériel et des demandes.

L'étude de l'existant a mis en évidence les limites de cette organisation — absence de vitrine numérique, traitement manuel des devis, gestion non centralisée du matériel de location — tout en identifiant ce qu'il fallait préserver : la relation de confiance et la personnalisation qui caractérisent le savoir-faire de l'entreprise. L'analyse des besoins qui a suivi a permis de traduire ce constat en fonctionnalités concrètes, réparties entre deux acteurs bien distincts : le visiteur, pour qui la plateforme devait offrir une vitrine claire et des outils de contact simples (devis, newsletter), et l'administrateur, pour qui elle devait offrir un espace de gestion centralisé et sécurisé.

Le cahier des charges a formalisé ces besoins en objectifs, modules fonctionnels et contraintes techniques, avant que la phase de conception — architecture générale en trois couches, structuration MVC du back-end, modélisation MERISE et UML — ne pose les bases techniques du système : entités, relations, diagrammes de cas d'utilisation, de classes, de séquence et d'activité. Cette double approche méthodologique a permis de passer d'un besoin exprimé en langage métier à un modèle exploitable pour le développement, sans perdre de vue la cohérence entre les données manipulées et les traitements réalisés.

La phase de réalisation a ensuite concrétisé l'ensemble de ces choix à travers une architecture Full-Stack JavaScript : une interface React.js moderne et responsive côté visiteur (présentation de l'entreprise, services, galerie de réalisations, catalogue de location, formulaire de devis), et une API REST sécurisée développée avec Node.js et Express.js, associée à une base de données relationnelle MySQL. La sécurité a été traitée de façon transversale, avec une authentification par JWT, un chiffrement des mots de passe via bcrypt, une gestion maîtrisée des fichiers importés grâce à Multer, et une intégration du service Brevo pour la gestion de la newsletter. Enfin, la phase de tests a permis de valider chaque fonctionnalité développée — tests fonctionnels, tests de l'API, validation de la base de données, vérifications responsive — avant de préparer les modalités de déploiement de la solution.

Au terme de ce projet, l'application développée répond à l'ensemble des objectifs fixés initialement : elle offre à Maison Yara une vitrine professionnelle capable d'améliorer sa visibilité numérique, elle facilite la relation avec ses clients grâce à un système de demande de devis centralisé, elle valorise son savoir-faire à travers une galerie dynamique, et elle simplifie son organisation interne grâce à un espace d'administration complet et sécurisé.

Au-delà de l'aspect technique, ce projet a également été l'occasion de mobiliser l'ensemble des compétences attendues d'un développeur web et web mobile : recueil et analyse d'un besoin client réel, conception d'une architecture cohérente, développement front-end et back-end, sécurisation d'une application, et validation par des tests rigoureux. Il constitue ainsi une démonstration concrète de la capacité à mener un projet informatique de bout en bout, depuis l'expression du besoin jusqu'à une solution opérationnelle.

Plusieurs pistes d'évolution restent envisageables pour accompagner la croissance future de Maison Yara : la mise en place d'un calendrier de disponibilité pour les réservations de

matériel, l'intégration d'un système de paiement en ligne pour les acomptes, le développement d'une application mobile dédiée, ou encore l'enrichissement du tableau de bord administrateur avec des statistiques d'activité plus détaillées. Ces perspectives confirment la pertinence de l'architecture modulaire retenue, pensée dès l'origine pour évoluer sans remise en cause profonde du système existant.